CONGRESO PROXY • WORKSHOP DE 4 HORAS

React + IA

Construye Atlas IA: una app en React con guías de viaje hechas por IA

Tadeo • Senior Fullstack Developer • GDG Hermosillo

```
function App() {
  return (
    <Chat ia />
  );
}
```

QUIÉN SOY

Hola, soy Tadeo

10+

años construyendo software

Senior Fullstack Developer

React

y todo el ecosistema JavaScript

TypeScript, Node.js y NestJS en el día a día

GDG

organizador en Hermosillo

Comunidad de developers locales

OBJETIVO DEL TALLER

Al terminar, tendrás Atlas IA funcionando: una app en React que usa una API real y genera guías de viaje con IA

1 • Fundamentos

2 • React en la práctica

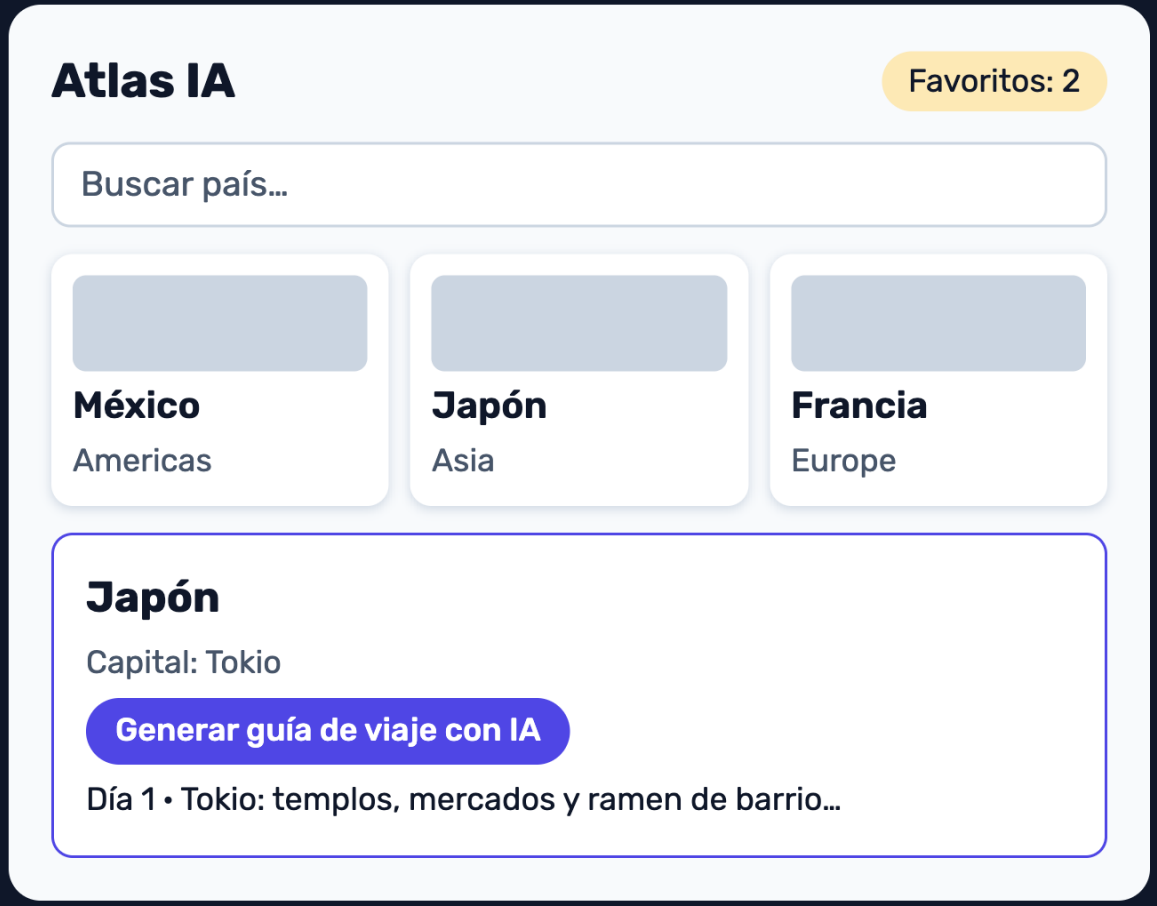
3 • Bonus: IA

EL PROYECTO

Vamos a construir Atlas IA

Un explorador de países con datos reales y una guía de viaje escrita por IA.

- ✓ Lista de países desde una API
- ✓ Buscador en vivo
- ✓ Favoritos con contador
- ✓ Panel de detalle por país
- ✓ **Guía de viaje generada con IA**



Congreso Proxy • Workshop React + IA • Maqueta ilustrativa del resultado final

CÓMO TRABAJAREMOS

Cada tema es un paso del proyecto

1 • Concepto

Un minuto de teoría, solo lo que necesitas para el paso.

2 • Código

Lo escribimos juntos en Atlas IA. Las líneas resaltadas son lo nuevo.

3 • Checkpoint

Comparas tu navegador con lo que debes ver antes de seguir.

```
$ git checkout paso-4
```

¿Te perdiste? Cada paso tiene su punto de control en el repositorio: https://github.com/TadeoZamora/Proxy-Atlas-workshop.git.

AGENDA

Cuatro horas, un proyecto

Hora	Bloque	Pasos de Atlas IA	Min
0:00	Bienvenida y proyecto base	Paso 0	15
0:15	Módulo 1 • Fundamentos de React	Pasos 1 a 5	80
1:35	Descanso	—	15
1:50	Módulo 2 • React en la práctica	Pasos 6 a 11	70
3:00	Módulo 3 • Bonus: IA	Pasos 12 a 16	50
3:50	Demos y cierre	Resultado final	10

PASO 0 • PROYECTO BASE

Crea Atlas IA

Necesitas:

- Node.js 20.6 o superior
- VS Code o tu editor favorito
- Un navegador con DevTools
- Una cuenta con API key de IA (Módulo 3)

CHECKPOINT

Ves 'Atlas IA' en http://localhost:5173

```
# Terminal
$ npm create vite@latest atlas-ia -- --template react
$ cd atlas-ia
$ npm install
$ npm run dev
```

```
// src/App.jsx: déjalo así
export default function App() {
  return <h1>Atlas IA</h1>;
}
```

MÓDULO 1 • 80 MIN

Fundamentos de React

01

Pasos 1 a 5 • Componentes, props, estado, listas y eventos: la interfaz de Atlas IA

MÓDULO 1

¿Qué es React y por qué usarlo?

React es una biblioteca de JavaScript para construir interfaces de usuario, creada en Meta.

Declarativo

Describes cómo debe verse la interfaz según los datos; React actualiza la pantalla por ti.

Componentes

Piezas reutilizables que se combinan para formar pantallas completas.

Ecosistema

Gran comunidad, miles de librerías y muchas oportunidades laborales.

PASO 1 • COMPONENTES Y JSX

Nuestro primer componente

- Un componente es una función que regresa JSX
- Su nombre empieza con mayúscula
- JSX es HTML dentro de JavaScript, con una sola raíz

```jsx
function PaisCard() {
  return (
    <div>
      <img
        src="https://flagcdn.com/w160/mx.png"
        alt="Bandera de México"
      />
      <h2>México</h2>
      <p>Americas · 128,455,567 hab.</p>
    </div>
  );
}

export default function App() {
  return (
    <div>
      <h1>Atlas IA</h1>
      <PaisCard />
    </div>
  );
}
```

CHECKPOINT

Ves el título Atlas IA y la tarjeta de México.

PASO 2 • PROPS

Tarjetas reutilizables

- Los props son los argumentos de un componente
- Bajan del padre al hijo y son de solo lectura
- Un componente, muchos países

```jsx
function PaisCard({ nombre, bandera, region, poblacion }) {
  return (
    <div>
      <img src={bandera} alt={`Bandera de ${nombre}`} />
      <h2>{nombre}</h2>
      <p>{region} · {poblacion.toLocaleString()} hab.</p>
    </div>
  );
}

// dentro de App:
<PaisCard
  nombre="México"
  bandera="https://flagcdn.com/w160/mx.png"
  region="Americas"
  poblacion={128455567}
/>
// repite con Japón (jp) y Francia (fr)
```

CHECKPOINT

Ves tres tarjetas distintas: México, Japón y Francia.

PASO 3 • USESTATE Y EVENTOS

Botón de favorito

- El estado es la memoria del componente
- Al cambiar, React vuelve a dibujarlo
- onClick recibe una función; no la ejecutes tú

```jsx
import { useState } from "react";

function PaisCard({ nombre, bandera, region, poblacion }) {
  const [esFavorito, setEsFavorito] = useState(false);

  return (
    <div>
      ...
      <p>{region} · {poblacion.toLocaleString()} hab.</p>
      <button onClick={() => setEsFavorito(!esFavorito)}>
        {esFavorito ? "Quitar favorito" : "Favorito"}
      </button>
    </div>
  );
}
```

CHECKPOINT

Al hacer clic, el botón alterna entre ‘Favorito’ y ‘Quitar favorito’, tarjeta por tarjeta.

PASO 4 • LISTAS Y KEY

Una lista de países

- .map() convierte datos en componentes
- Cada elemento lleva una key única: usa un id, no el índice
- Los datos tienen la misma forma que la API real

```
const PAISES = [
  {
    cca3: "MEX",
    name: { common: "México" },
    flags: { png: "https://flagcdn.com/w160/mx.png" },
    region: "Americas",
    population: 128455567,
  },
  // ...Japón, Francia y los que quieras
];

{PAISES.map((p) => (
  <PaisCard
    key={p.cca3}
    nombre={p.name.common}
    bandera={p.flags.png}
    region={p.region}
    poblacion={p.population}
  />
))}
```

CHECKPOINT

Ves una tarjeta por cada país del arreglo. Agrega otro y aparece sola.

PASO 5 • EVENTOS Y CONDICIONALES

Buscador en vivo

- Input controlado: su valor vive en el estado
- onChange actualiza el estado en cada tecla
- Condicional: sin resultados, mostramos un aviso

```
const [busqueda, setBusqueda] = useState("");

const visibles = PAISES.filter((p) ⇒
  p.name.common
    .toLowerCase()
    .includes(busqueda.toLowerCase())
);

<input
  value={busqueda}
  onChange={(e) ⇒ setBusqueda(e.target.value)}
  placeholder="Buscar país…"
/>

{visibles.length ⩵ 0 ? (
  <p>No hay resultados</p>
) : (
  visibles.map((p) ⇒ <PaisCard key={p.cca3} ... />)
)}
```

CHECKPOINT

Escribes ‘jap’ y solo queda Japón. Con ‘zzz’ ves ‘No hay resultados’.

TU TURNO • 15 MIN

Filtro por región

- Agrega al menos cinco países de distintas regiones a PAISES
- Crea un select con Todas, Americas, Europe y Asia
- Combina región y búsqueda dentro de visibles
- Reto: un botón para limpiar los filtros

useState onChange filter



DESCANSO

15 minutos

Regresamos con el Módulo 2: useEffect, APIs y estilos.

MÓDULO 2 • 70 MIN

React en la práctica

02

Pasos 6 a 11 • useEffect y fetch, componentes en archivos y estilos: Atlas IA con datos reales

PASO 6 • USEEFFECT Y FETCH

Datos reales de una API

- useEffect corre después de que React pinta la pantalla
- El arreglo [] significa: solo una vez, al montar
- fetch pide los datos; guárdalos en el estado

```
const API =
  "https://restcountries.com/v3.1/all" +
  "?fields=name,cca3,flags,region,population,capital";

const [paises, setPaises] = useState([]);

useEffect(() => {
  async function cargar() {
    const res = await fetch(API);
    const datos = await res.json();
    setPaises(datos);
  }
  cargar();
}, []);

// ahora filtra paises, ya no PAISES
const visibles = paises.filter(...);
```

CHECKPOINT

En vez de cinco países, ves más de 200 con sus banderas reales.

PASO 7 • CARGANDO Y ERROR

Los tres estados de una petición

- Cargando, éxito o error: la UI cubre los tres
- try / catch / finally maneja los fallos
- Retorno temprano: un if con return

```
const [cargando, setCargando] = useState(true);
const [error, setError] = useState(null);

useEffect(() ⇒ {
  async function cargar() {
    try {
      const res = await fetch(API);
      if (!res.ok) throw new Error("Error " + res.status);
      setPaises(await res.json());
    } catch (e) {
      setError(e.message);
    } finally {
      setCargando(false);
    }
  }
  cargar();
}, []);

if (cargando) return <p>Cargando países…</p>;
if (error) return <p>Error: {error}</p>;
```

CHECKPOINT

Ves ‘Cargando países…’ un instante. Con una URL mala, ves el error.

PASO 8 • ORGANIZA TU CÓDIGO

Componentes en archivos

- Un archivo por componente en src/components: Header, Buscador, ListaPaises y PaisCard
- Si varios necesitan un dato, súbelo al padre
- El padre baja datos y funciones por props

CHECKPOINT

Todo funciona igual y el encabezado cuenta tus favoritos.

```
// App.jsx: los favoritos viven aquí
const [favoritos, setFavoritos] = useState([]);

function toggleFavorito(codigo) {
  setFavoritos((prev) ⇒
    prev.includes(codigo)
      ? prev.filter((c) ⇒ c ≠ codigo)
      : [...prev, codigo]
  );
}

<Header totalFavoritos={favoritos.length} />
<ListaPaises
  paises={visibles}
  favoritos={favoritos}
  onToggle={toggleFavorito}
/>

// PaisCard: sin estado propio, recibe esFavorito y onToggle
```

PASO 9 • PANEL DE DETALLE

Selecciona un país

- seleccionado guarda el país elegido, o null
- && muestra el panel solo si hay selección
- El hijo avisa al padre con una función en props

```jsx
// App.jsx
const [seleccionado, setSeleccionado] = useState(null);

{seleccionado && (
  <DetallePais pais={seleccionado} />
)}

<ListaPaises ... onSelect={setSeleccionado} />

// components/DetallePais.jsx
export default function DetallePais({ pais }) {
  return (
    <section>
      <h2>{pais.name.common}</h2>
      <p>Capital: {pais.capital?.[0] ?? "Sin capital"}</p>
      <p>Población: {pais.population.toLocaleString()}</p>
    </section>
  );
}
```

CHECKPOINT

Clic en ‘Ver detalle’ y aparece el panel con capital y población.

PASO 10 • ESTILOS CON TAILWIND

Instala Tailwind

- CSS tradicional: un archivo .css y className
- Tailwind: clases utilitarias directo en el JSX
- Con Vite se instala como plugin

```
# 1. Terminal
$ npm install tailwindcss @tailwindcss/vite

// 2. vite.config.js
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});

/* 3. src/index.css: reemplaza todo su contenido */
@import "tailwindcss";
```

CHECKPOINT

Reinicias npm run dev y los títulos pierden su tamaño: Tailwind ya está activo.

PASO 11 • CLASES DE TAILWIND

Dale estilo a la tarjeta

- Una clase, una propiedad: p-4 es padding
- hover: y md: agregan estados y tamaños
- className acepta cualquier expresión de JavaScript

```jsx
// PaisCard.jsx
<div className="rounded-2xl bg-white p-4 shadow">
  <img
    className="h-28 w-full rounded-lg object-cover"
    src={bandera}
    alt={`Bandera de ${nombre}`}
  />
  <h2 className="mt-3 text-lg font-semibold">{nombre}</h2>
  <button
    className={`rounded-full px-3 py-1 ${
      esFavorito ? "bg-amber-400" : "bg-slate-200"
    }`}
    onClick={onToggle}
  >

// ListaPaises.jsx
<div className="grid grid-cols-2 gap-4 md:grid-cols-4">
```

CHECKPOINT

Cuadrícula de tarjetas con sombra y favoritos en amarillo.

TU TURNO • 10 MIN

Solo favoritos y tu estilo

- Agrega un botón o checkbox ‘Solo favoritos’ con useState
- Filtra visibles usando la lista de favoritos
- Cambia colores, tipografía y espacios a tu gusto con Tailwind
- Reto: un botón de modo oscuro

Estado en el padre | Props | Tailwind

MÓDULO 3 • BONUS • 50 MIN

IA en tu aplicación

03

Pasos 12 a 16 • Un servidor propio y la guía de viaje generada por IA dentro de Atlas IA

MÓDULO 3 • BONUS

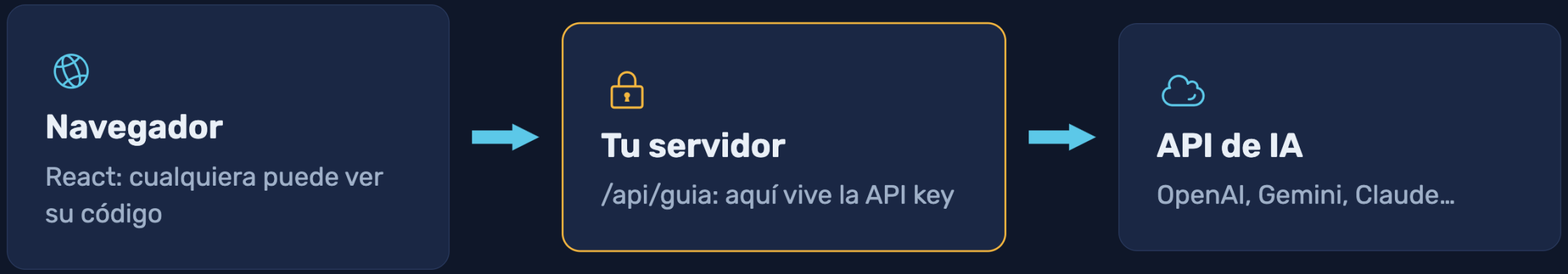
¿Qué APIs de IA podemos usar hoy?

Proveedor	Cómo empezar	Lo que ofrece
OpenAI	Plataforma de OpenAI	Modelos GPT para texto, imagen y audio
Google Gemini	Google AI Studio	Suele tener capa gratuita para empezar
Anthropic	Claude Console	Claude para texto y visión
Modelos locales	Ollama	Corren en tu computadora, sin API key

Revisa siempre precios, cuotas y condiciones vigentes en la documentación de cada proveedor.

MÓDULO 3 • SEGURIDAD

La API key nunca va en el frontend



Todo lo que está en el código de React lo puede ver cualquiera desde el navegador.

PASO 12 • SERVIDOR BASE

Tu propio servidor

- Una carpeta server/ con Express
- La API key vive en un archivo .env, nunca en React
- El SDK de la IA se crea una sola vez

```
# Terminal
$ npm install express @google/genai
// package.json, dentro de "scripts"
  "server": "node --env-file=.env server/index.js"
// server/index.js
import express from "express";
import { GoogleGenAI } from "@google/genai";
const app = express();
app.use(express.json());
const ai = new GoogleGenAI({
  apiKey: process.env.GEMINI_API_KEY,
});
// aquí irá el endpoint (paso 13)
app.listen(3001, () ⇒ console.log("Servidor listo"));
# .env (no lo subas a git)
GEMINI_API_KEY=tu_clave
MODELO=nombre-del-modelo
```

CHECKPOINT

Corres npm run server y la terminal dice 'Servidor listo'.

PASO 13 • EL ENDPOINT

Una ruta para la guía

- app.post define una ruta que recibe JSON
- El prompt se arma aquí, con el país recibido
- La key nunca sale del servidor

```
// server/index.js, antes de app.listen
app.post("/api/guia", async (req, res) => {
  try {
    const { pais } = req.body;
    const prompt =
      `Guía de viaje de 3 días para ${pais}. ` +
      "Incluye qué comer y un consejo local. " +
      "Responde en español.";
    const r = await ai.models.generateContent({
      model: process.env.MODELO,
      contents: prompt,
    });
    res.json({ guia: r.text });
  } catch {
    res.status(500).json({ error: "Falló la IA" });
  }
});
```

CHECKPOINT

Reinicias el servidor sin errores y queda esperando peticiones.

PASO 14 • PROXY DE VITE

Conecta React con tu servidor

- React corre en el puerto 5173 y el servidor en el 3001
- El proxy reenvía /api al servidor
- Dos terminales: una para cada proceso

```
// vite.config.js
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    proxy: { "/api": "http://localhost:3001" },
  },
});

# Terminal 1: el servidor
$ npm run server

# Terminal 2: la app de React
$ npm run dev
```

CHECKPOINT

Servidor y app corren a la vez, sin errores en la consola.

PASO 15 • FETCH CON POST

Pídele la guía a la IA

- Mismo patrón del paso 7: cargando, error y datos
- POST con el país en JSON hacia tu propio endpoint
- Nunca llamas a la IA directo desde el navegador

```
export default function GuiaIA({ pais }) {
  const [guia, setGuia] = useState("");
  const [cargando, setCargando] = useState(false);
  const [error, setError] = useState(null);

  async function generar() {
    setCargando(true);
    setError(null);
    try {
      const res = await fetch("/api/guia", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ pais: pais.name.common }),
      });
      if (!res.ok) throw new Error();
      const datos = await res.json();
      setGuia(datos.guia);
    } catch {
      setError("No se pudo generar la guía");
    } finally {
      setCargando(false);
    }
  }
```

CHECKPOINT

El proyecto compila sin errores. Aún no hay botón: lo agregamos en el paso 16.

PASO 16 • LA INTERFAZ

Muestra la guía

- El botón se desactiva mientras carga
- Tres casos: error, guía o nada
- key={pais.cca3} reinicia el estado al cambiar de país

```
// GuiaIA.jsx: el return
return (
  <div>
    <button onClick={generar} disabled={cargando}>
      {cargando ? "Generando…" : "Generar guía con IA"}
    </button>
    {error && <p>{error}</p>}
    {guia && <p className="whitespace-pre-line">{guia}</p>}
  </div>
);

// DetallePais.jsx: dentro del section
<GuiaIA pais={pais} />

// App.jsx: reinicia el estado al cambiar de país
<DetallePais key={seleccionado.cca3} pais={seleccionado} />
```

CHECKPOINT FINAL

Eliges un país, pulsas el botón y la IA te escribe su guía de viaje.

TU TURNO • 10 MIN

Personaliza la guía

- Agrega un select de tono: mochilero, familiar o lujo
- Envíalo en el body y súmalo al prompt del servidor
- Agrega un botón para copiar la guía
- Reto: una pregunta de seguimiento, como un mini chat

Estado | fetch POST | Prompt

RESULTADO FINAL

Atlas IA, completo

Tema	Pasos	Dónde lo usaste
Componentes y JSX	1, 8	PaisCard, Header, ListaPaises
Props	2, 8, 9	Datos y funciones del padre al hijo
Estado con useState	3, 5, 8	Favoritos, búsqueda y selección
Condicionales y listas	4, 5, 7	.map con key y ‘No hay resultados’
Eventos	3, 5, 9	Clics e input controlado
useEffect y fetch	6, 7	Países desde una API real
Organización y estilos	8, 10, 11	Carpeta components y Tailwind
IA con React	12 a 16	Servidor, endpoint y componente GuiaIA

CIERRE

Lo que sigue

Ideas para seguir

- Guardar favoritos en localStorage
- Respuestas de la IA en streaming
- Un mini chat por país
- Rutas con React Router
- Publicar frontend y servidor en la nube

Dónde aprender más

- react.dev: documentación oficial de React
- vite.dev: herramienta de desarrollo
- tailwindcss.com: guía de Tailwind
- Documentación de tu proveedor de IA
- Comunidades locales como GDG

CONGRESO PROXY

¡Gracias!

Construiste Atlas IA de principio a fin. El siguiente paso es tuyo.

https://github.com/TadeoZamora/Proxy-Atlas-workshop.git •
www.linkedin.com/in/xtadeozamora IG: @XPresidenteX